

VaultFS

Federated Knowledge and Resource Repository

Overview

VaultFS is a federated knowledge and resource management platform that combines hierarchical content organization, personal knowledge management, media consumption, and distributed storage pooling.

The platform allows users to organize resources through a folder-based structure while transparently distributing storage across multiple Cloudinary accounts contributed by trusted users.

Unlike traditional cloud storage platforms, VaultFS separates metadata management from physical storage and introduces a trust-based storage federation model that enables storage sharing without requiring centralized infrastructure.

The project is designed to demonstrate concepts from:

- Distributed Systems
- Database Design
- Storage Systems
- Graph Theory
- Full Stack Engineering
- System Design

while remaining deployable entirely on free-tier infrastructure.

Problem Statement

Modern information is scattered across multiple platforms:

- Notes in note-taking applications
- PDFs in cloud storage
- Videos in media platforms
- Bookmarks in browsers
- Learning resources across multiple websites

Managing these resources becomes increasingly difficult as collections grow.

Additionally, free cloud storage platforms impose strict storage limitations.

VaultFS addresses both problems by providing:

1. Unified resource organization.
 2. Federated storage expansion through trusted users.
-

Core Objectives

The platform must support:

- Hierarchical folders
 - Notes
 - PDFs
 - Images
 - Videos
 - External links
 - YouTube resources
 - Global search
 - Resource sharing
 - Multi-user collaboration
 - Distributed storage pooling
-

Design Philosophy

The system is built around three principles:

Metadata First

Metadata is the primary asset.

Files are treated as resources attached to metadata.

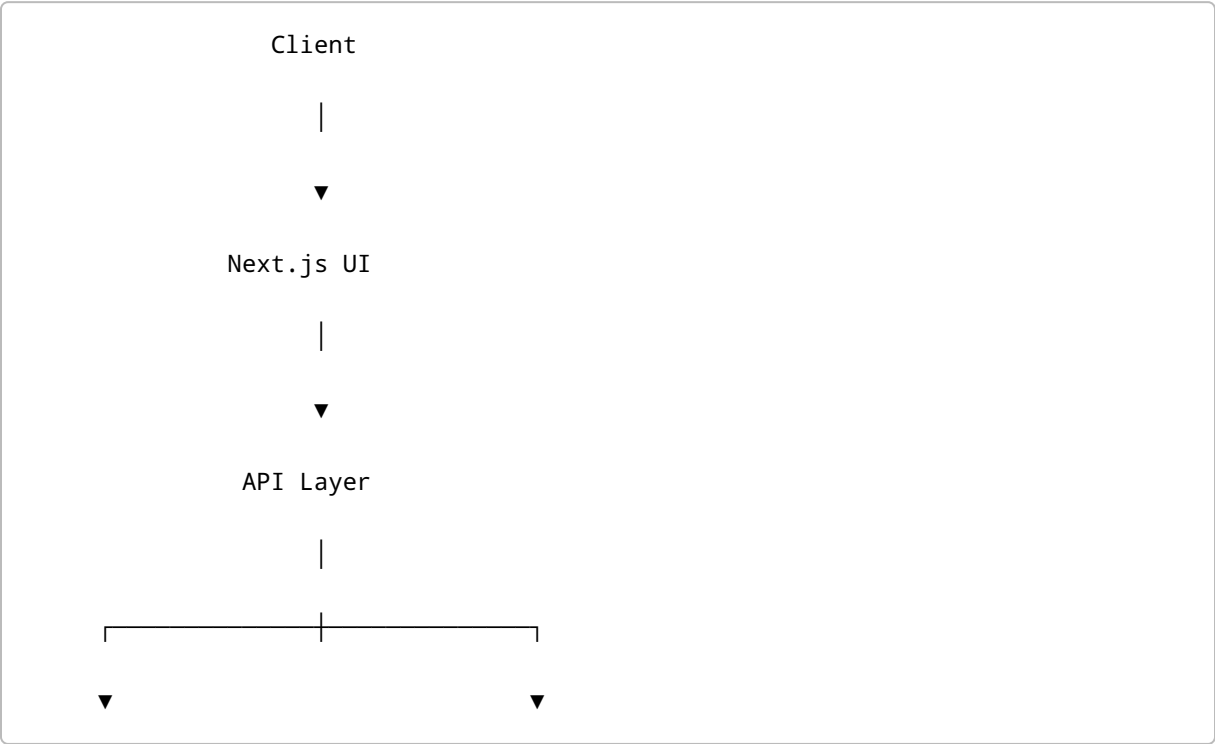
Storage Independence

Storage providers are abstracted from application logic.

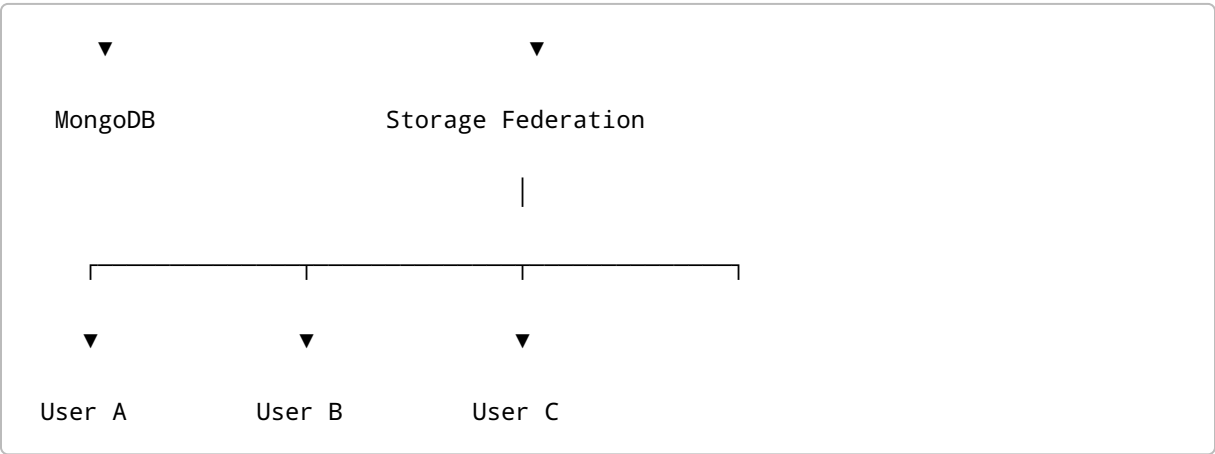
Federated Expansion

Storage capacity grows through trusted users contributing storage nodes.

High-Level Architecture



Metadata Service Storage Service



Cloudinary A1 Cloudinary B1 Cloudinary C1

Cloudinary A2 Cloudinary B2 Cloudinary C2

System Components

Frontend

Responsibilities:

- Folder navigation
- Resource browsing
- Search
- Uploads
- Sharing
- Authentication

Technology:

- Next.js
- Tailwind CSS
- Server Components

Design Constraints:

- Low memory consumption
 - Server-side rendering
 - Minimal client state
 - Lazy loading
-

API Layer

Responsibilities:

- Authentication
- Folder operations
- Resource operations
- Search
- Sharing
- Storage routing

Technology:

- Next.js Route Handlers

The API remains stateless.

Metadata Service

The Metadata Service is the source of truth.

Stores:

- Users
- Folders
- Resources
- Storage Nodes
- Permissions
- Tags
- Trust Relationships

Technology:

- MongoDB Atlas
-

Storage Service

The Storage Service manages storage allocation.

Responsibilities:

- Storage discovery
- Node selection
- Upload orchestration
- Storage accounting
- Health monitoring

The service never stores file contents.

Federated Storage Model

Personal Storage

Every user owns one or more storage nodes.

Example:

User A

- Cloudinary A1

- Cloudinary A2

User B

- Cloudinary B1

User C

- Cloudinary C1
- Cloudinary C2

A storage node represents a single Cloudinary account.

Storage Federation

Users may establish trusted storage relationships.

Example:

User A trusts User B.

User B trusts User A.

Both users may contribute storage to a shared storage pool.

This process is called a Storage Handshake.

Trust Graph

The platform models storage relationships as a graph.

Example:

A — B

|

C

Storage sharing occurs only through explicitly approved relationships.

No storage is shared automatically.

Storage Node Model

Each node contains:

```
{ "_id": "...",  
  
  "ownerId": "...",  
  
  "provider": "cloudinary",  
  
  "active": true,  
  
  "allocatedStorage": 0 }
```

A node belongs to exactly one owner.

Storage Discovery

When a user uploads a file:

The Storage Service discovers:

1. User-owned nodes.
2. Trusted shared nodes.

Example:

User A Upload

Available Nodes:

- A1
- A2
- B1

Storage Service selects one node.

Storage Allocation Strategy

Version 1 uses:

Round Robin Allocation

Example:

Upload 1 → A1

Upload 2 → B1

Upload 3 → A2

Upload 4 → A1

Advantages:

- Predictable
- Simple
- Low overhead

Future versions may use:

- Capacity-aware balancing
- Weighted allocation
- Consistent hashing

Knowledge Model

Everything inside VaultFS is treated as a Resource.

Folder

Container resource.

Provides hierarchy.

Note

Markdown-based content.

Stored entirely inside MongoDB.

File

Generic uploaded content.

PDF

Document resource with integrated viewing.

Video

Media resource streamed directly from Cloudinary.

Image

Image resource.

YouTube Resource

Embedded external video.

External Link

Reference to external content.

Folder Architecture

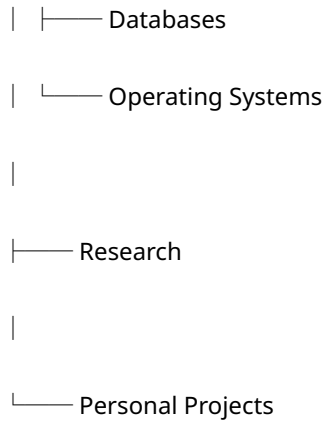
Folders are represented as a tree.

Example:

/

| — Computer Science

| | — Networking



Folders may contain:

- Resources
- Subfolders

Resource Metadata

Every resource contains:

```
{ "_id": "...",  
  "name": "...",  
  "type": "...",  
  "ownerId": "...",  
  "folderId": "...",  
  "tags": [],  
  "description": "",  
  "storageNodeId": "..." }
```

This metadata powers discovery and search.

Knowledge Management Features

Resources support:

- Tags
- Descriptions
- Categories
- References

Example:

TCP Notes

Tags:

- networking
- interview
- transport-layer

This transforms storage into searchable knowledge.

Search System

Search operates entirely on metadata.

Searchable fields:

- Resource names
- Folder names
- Note content
- Tags
- Descriptions

Implementation:

MongoDB Text Indexes

Advantages:

- Fast
 - Lightweight
 - No additional infrastructure
-

Multi-User Collaboration

Resources support three visibility levels.

Private

Only owner can access.

Shared

Specific users may access.

Public

Anyone may access.

Storage Accounting

The platform tracks:

Provided Storage

Consumed Storage

Available Storage

Example:

User A

Provides: 30 GB

Consumes: 12 GB

User B

Provides: 10 GB

Consumes: 25 GB

This enables future quota systems.

Performance Requirements

The system must remain usable on low-memory hardware.

Requirements:

- No client-side indexing
- No large frontend state stores
- No file buffering on server
- No eager loading of folders
- No full PDF loading

Techniques:

- SSR
 - Pagination
 - Lazy loading
 - Direct streaming
 - Incremental data fetching
-

Security Model

Authentication:

JWT or NextAuth

Storage Sharing:

Explicit handshake approval required.

Storage Ownership:

Always retained by original owner.

No automatic storage exposure.

Distributed Systems Concepts Demonstrated

The project intentionally incorporates:

- Federated Architecture

- Storage Pooling
 - Storage Routing
 - Metadata Separation
 - Resource Allocation
 - Graph-Based Trust Networks
 - Capacity Tracking
 - Horizontal Storage Expansion
-

Version 1 Scope

Features included:

- Authentication
 - Folder Hierarchy
 - Notes
 - Tags
 - File Uploads
 - Cloudinary Storage Nodes
 - Storage Handshakes
 - Storage Federation
 - PDF Viewer
 - Video Streaming
 - Image Viewing
 - YouTube Resources
 - Search
 - Sharing Permissions
 - Storage Statistics
-

Future Versions

Potential extensions:

- Node Health Monitoring
 - Replication
 - Deduplication
 - Consistent Hashing
 - Versioned Notes
 - Audit Logs
 - Resource Recommendations
 - AI-Assisted Search
 - Alternative Storage Providers
-

Educational Outcomes

Completion of VaultFS demonstrates practical understanding of:

Database Systems

- Hierarchical Data Structures
- Indexing
- Metadata Design

Distributed Systems

- Federation
- Resource Routing
- Storage Allocation
- Capacity Management

System Design

- Scalable Architectures
- Service Separation
- Multi-Tenant Systems

Software Engineering

- Interface Design
- Modular Architecture
- Extensible Systems

The resulting platform functions as a distributed knowledge repository whose storage capacity expands through trusted user relationships while maintaining centralized metadata management and low operational cost.